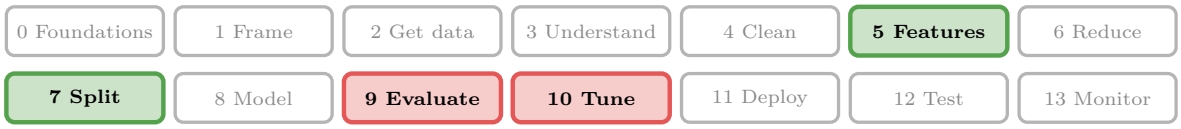
Machine Learning Pipelines

Contents

1. Overview . . . 3
2. What a pipeline is . . . 4
   - 2.1 Why chaining the steps matters . . . 4
3. The Titanic data . . . 5
   - 3.1 What the columns need . . . 5
4. Without a pipeline . . . 6
   - 4.1 Filling the missing values . . . 6
   - 4.2 One-hot encoding Sex and Embarked . . . 7
   - 4.3 Joining the arrays and training . . . 8
   - 4.4 Predicting for a new passenger . . . 9
   - 4.5 What goes wrong without a pipeline . . . 10
5. Building a pipeline . . . 11
   - 5.1 The plan . . . 11
   - 5.2 Step 1: imputation . . . 11
   - 5.3 Positions refer to the previous step's output . . . 12
   - 5.4 Step 2: one-hot encoding . . . 13
   - 5.5 Step 3: scaling . . . 13
   - 5.6 Step 4: feature selection . . . 13
   - 5.7 Step 5: the model . . . 15
   - 5.8 Chaining the steps with Pipeline . . . 15
   - 5.9 Pipeline compared with make_pipeline . . . 15
6. Training and predicting with a pipeline . . . 16
   - 6.1 One new passenger through the pipeline . . . 16
   - 6.2 Pipelines without a model . . . 17
7. Looking inside a pipeline . . . 18
   - 7.1 The pipeline as a diagram . . . 18
   - 7.2 Pulling out fitted steps . . . 18
8. Cross-validation with a pipeline . . . 19
   - 8.1 Why the preprocessing must be inside: data leakage . . . 20
9. Hyperparameter tuning with a pipeline . . . 22
10. Exporting the pipeline for production . . . 23
    - 10.1 Using column names instead of positions . . . 25

11. Summary 26
12. Sources 26
13. Key terms 27

Where this fits: Pipeline map steps 5 (Engineer features), 7 (Split), 9 (Evaluate), 10 (Tune). See the Course map.



- **Builds on:** Model-based learning; Overfitting; Deployment; Saving models with pickle; Column transformer.
- **Leads to:** Feature construction and splitting; K-nearest neighbours; Voting ensembles; Random forest; AdaBoost; Grid and random search; Stacking and blending; Random under- and oversampling; Optuna; Improving a neural network; Keras Tuner.
- **Compare with:** Train-test split; OOB score.

1. Overview

Key point: A **pipeline** (G-1498) chains every preprocessing step and the model into one object, so the same steps run in the same order on the training data, the test data and every new input.

Figure 1 shows the whole topic. Five steps live inside one object called `pipe`:

1. fill missing values;
2. one-hot encode;
3. scale;
4. keep the best columns;
5. train a **decision tree** (G-561).

Calling `pipe.fit` sends the 712 training **observations** (G-1374) (records, one row each) through every step; calling `pipe.predict` sends one new passenger through the same fitted steps and gives a prediction.

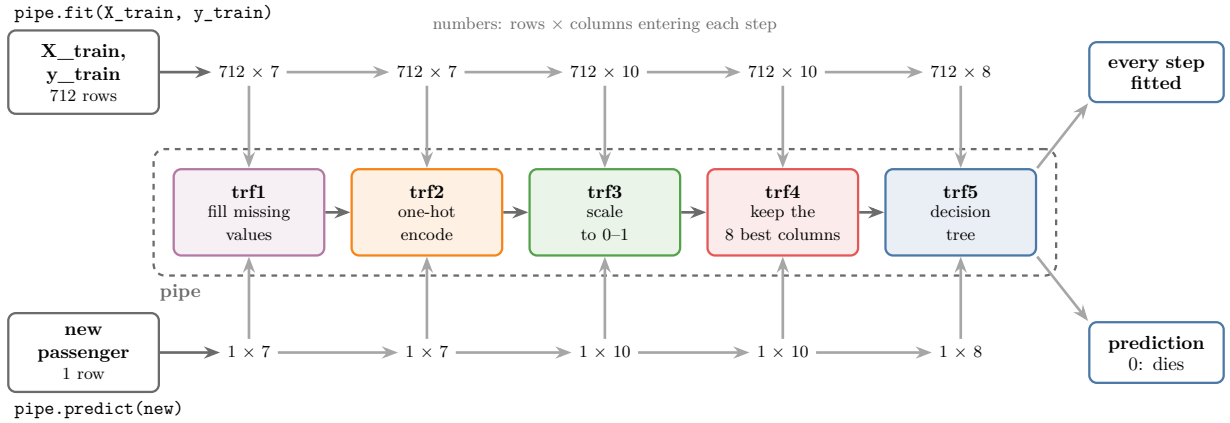


Figure 1: One pipeline object: fit on the training data, then predict for one new passenger with the same fitted steps

2. What a pipeline is

> **Key point:** In a **pipeline** (G-1498), the output of each step is the input of the next one, and scikit-learn passes the data along by itself.

A **pipeline** (G-1499) is a mechanism that chains several steps together, so that the output of each step is used as the input to the next. Think of a car assembly line: each station does one job and passes the car on, and every car goes through the same stations in the same order. We give the data to the first step and take the result from the last one; everything in between is handled internally.

Suppose a dataset has missing values and categorical **features** (G-772) (input variables, one column each of the data table). Building a model then takes three steps:

1. Fill the missing values.
2. Encode the categorical columns.
3. Train the model.

Done separately, these are three pieces of code. With a pipeline they become one object: we give it the raw input, and it returns the output.

2.1 Why chaining the steps matters

> **Key point:** Every input the model ever sees needs exactly the same preprocessing, and a **pipeline** (G-1498) guarantees that.

The training set, the test set and every future input must go through the same preprocessing. The future inputs are the important case: once a model is **deployed** (G-592) to a server, for example behind a website, each new input arrives raw. A raw input may have missing values to fill and categories to encode before the model can use it.

Without a **pipeline** (G-1499), every preprocessing step must be written a second time in the website's code. With a pipeline, the steps travel with the model, so nothing has to be repeated.

To see the difference, we build the same small project twice: first without a pipeline, then with one.

3. The Titanic data

> **Key point:** 891 passengers; we predict `Survived` (1 = survived, 0 = died) from seven features, two of which have missing values.

The data is Kaggle's Titanic training file: one row per passenger on the Titanic, so each passenger is one **observation** (G-1374). We drop four columns that are not useful here: `PassengerId`, `Name`, `Ticket` and `Cabin`. Eight columns remain.

Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.25	S
1	1	female	38.0	1	0	71.28	C
1	3	female	26.0	0	0	7.93	S

The columns mean:

- **Survived:** the **target** (G-1949), the output we predict. 1 if the passenger survived, 0 if not.
- **Pclass:** ticket class, 1, 2 or 3.
- **Sex:** male or female.
- **Age:** age in years.
- **SibSp:** number of siblings and spouses on board.
- **Parch:** number of parents and children on board.
- **Fare:** ticket price.
- **Embarked:** port where the passenger boarded: C (Cherbourg), Q (Queenstown) or S (Southampton).

The other seven columns are the **features** (G-772). The task is a model that takes a passenger's seven features and predicts whether that passenger survives.

3.1 What the columns need

> **Key point:** Age and Embarked have missing values; Sex and Embarked are nominal text.

`df.isnull().sum()` shows two columns with missing values: Age (177 missing) and Embarked (2 missing). We cannot train a model until they are filled.

Sex and Embarked hold text with no order, so they need **one-hot encoding** (G-1379; one column per category): one 0/1 column per category. The other columns are already numbers.

Figure 2 sorts the seven **features** (G-772) by the work they need. Watch Embarked: it is the one column that needs both jobs.

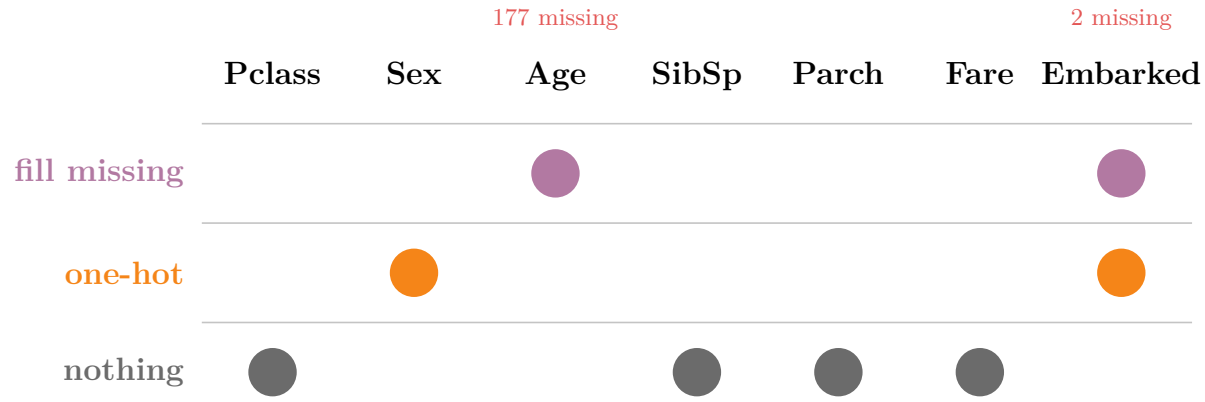


Figure 2: What each of the seven features needs before a model can use it

Python: Loading the data, dropping columns and splitting.

```python
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/titanic_train.csv")
df = df.drop(columns=["PassengerId", "Name",
                      "Ticket", "Cabin"])
df.isnull().sum()   # Age: 177, Embarked: 2

X_train, X_test, y_train, y_test = train_test_split(
    df.drop(columns=["Survived"]),  # 7 inputs
    df["Survived"],                 # target
    test_size=0.2, random_state=42)
# X_train: (712, 7), X_test: (179, 7)
```

4. Without a pipeline

Key point: By hand, every step needs its own object, its own arrays for the training and test sets, and a final join.

4.1 Filling the missing values

Key point: Age gets the mean age; Embarked gets the most frequent port, S.

The two columns need different imputers:

- **Age:** a `SimpleImputer` (the scikit-learn class that fills gaps; imputing numerical data) with its default setting fills each gap with the mean age of the training set.

- **Embarked:** a `SimpleImputer(strategy="most_frequent")` fills each gap with the most common port. In the training set that is S (Southampton).

Python: One imputer per column.

```
from sklearn.impute import SimpleImputer

si_age = SimpleImputer()
si_embarked = SimpleImputer(strategy="most_frequent")

X_train_age = si_age.fit_transform(X_train[["Age"]])
X_train_embarked = si_embarked.fit_transform(
    X_train[["Embarked"]])

X_test_age = si_age.transform(X_test[["Age"]])
X_test_embarked = si_embarked.transform(
    X_test[["Embarked"]])
```

4.2 One-hot encoding Sex and Embarked

Key point: Sex becomes 2 columns and Embarked 3; Embarked is encoded from its imputed array, not the original column.

Sex and Embarked each get their own `OneHotEncoder`. The two cannot share one encoder here: the imputed Embarked is now a separate array, while Sex is still in `X_train`. The original Embarked column in `X_train` still has its missing values.

Two settings matter:

- **`handle_unknown="ignore"`** (G-876): if a later input has a category never seen in training, the encoder outputs all zeros for it instead of raising an error.
- **No `drop="first"`:** we keep every column. Dropping one avoids **multicollinearity** (G-1273; inputs that depend on each other), which matters for linear models, but a **decision tree** (G-561) is not affected by it.

Python: One encoder per column.

```
from sklearn.preprocessing import OneHotEncoder

ohe_sex = OneHotEncoder(sparse_output=False,
                        handle_unknown="ignore")
ohe_embarked = OneHotEncoder(sparse_output=False,
                             handle_unknown="ignore")

X_train_sex = ohe_sex.fit_transform(X_train[["Sex"]])
X_train_embarked = ohe_embarked.fit_transform(
    X_train_embarked)   # the imputed array
```

```
X_test_sex = ohe_sex.transform(X_test[["Sex"]])
X_test_embarked = ohe_embarked.transform(X_test_embarked)
```

4.3 Joining the arrays and training

> **Key point:** Four untouched columns, plus Age, plus 2 Sex columns, plus 3 Embarked columns: 10 columns.

The four columns that needed nothing (Pclass, SibSp, Parch, Fare) are taken out with `drop`. Then `np.concatenate` joins everything side by side, for both sets. The four untouched columns, Age, the 2 Sex columns and the 3 Embarked columns add up to 10 columns:

$$4 + 1 + 2 + 3 = 10$$

A **decision tree** (G-561) (a model that predicts by a chain of yes/no questions) is trained on the result. On the test set it gets an **accuracy** (G-162; the share of correct predictions) of 79.3%.

Figure 3 draws the whole manual route. Watch how many separate objects and arrays feed the one join at the end.

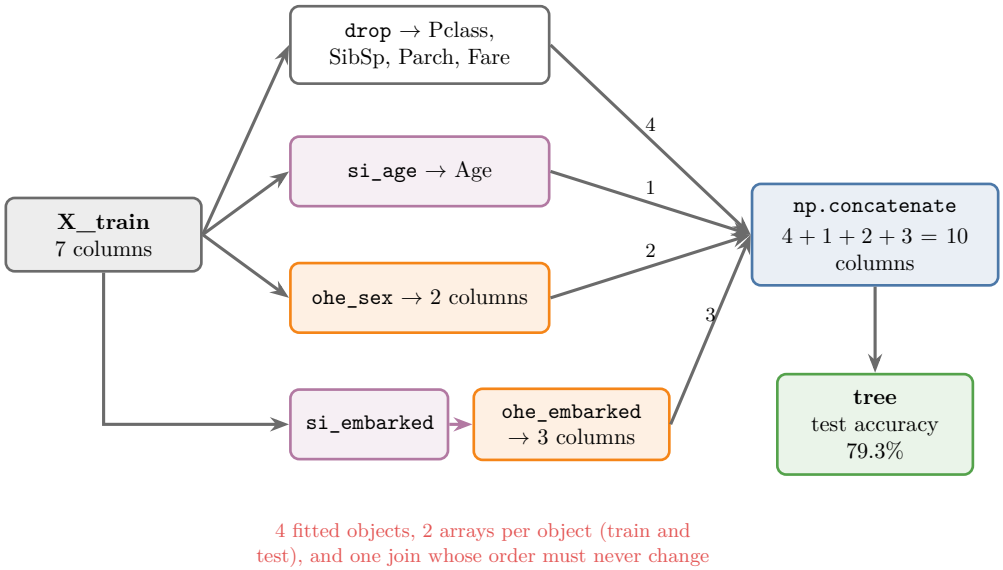


Figure 3: Preprocessing by hand: four fitted objects, four arrays and one join into 10 columns

> **Python:** Joining and training.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
X_train_rem = X_train.drop(columns=["Sex", "Age",
                                    "Embarked"])
X_test_rem = X_test.drop(columns=["Sex", "Age",
                                  "Embarked"])
X_train_transformed = np.concatenate((X_train_rem,
    X_train_age, X_train_sex, X_train_embarked), axis=1)
X_test_transformed = np.concatenate((X_test_rem,
    X_test_age, X_test_sex, X_test_embarked), axis=1)
# (712, 10) and (179, 10)

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train_transformed, y_train)
accuracy_score(y_test, clf.predict(X_test_transformed))
# 0.793
```

A **column transformer** (G-415) would have made this part shorter, but the next problem remains even with one.

4.4 Predicting for a new passenger

> **Key point:** The website's code must load every fitted encoder and repeat every training step, in the same order.

Suppose the model goes behind a website. A visitor types in a new passenger's details, and the model says whether that passenger survives. For this, the website's code needs more than the trained tree.

The fitted objects are saved to files with **pickle** (G-1494; saving a model), Python's tool for writing an object to a file and reading it back later:

- **`clf.pkl`:** the trained **decision tree** (G-561).
- **`ohe_sex.pkl` and `ohe_embarked.pkl`:** the two fitted encoders. A new input arrives as "male" and "S", but the tree only understands numbers, so the website must encode them exactly as in training.

The imputers are not saved. The website's form will not accept an empty field, so a new input never has missing values. Saving them would not hurt, but they would never be used.

Python: Saving the fitted objects.

```
import pickle

pickle.dump(ohe_sex, open("models/ohe_sex.pkl", "wb"))
pickle.dump(ohe_embarked,
            open("models/ohe_embarked.pkl", "wb"))
pickle.dump(clf, open("models/clf.pkl", "wb"))
```

`"wb"` opens the file for writing in binary; `"rb"` opens it for reading. A `.pkl` file is binary:

we cannot read it, only load it.

The website's code then loads the three files and rebuilds the 10 columns for the new passenger: second class, male, 31 years old, no siblings, spouse, parents or children on board, fare 10.5, boarded at Southampton.

Python: The website's code without a **pipeline** (G-1498).

```
ohe_sex = pickle.load(open("models/ohe_sex.pkl", "rb"))
ohe_embarked = pickle.load(
    open("models/ohe_embarked.pkl", "rb"))
clf = pickle.load(open("models/clf.pkl", "rb"))

# Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
new = pd.DataFrame([[2, "male", 31.0, 0, 0, 10.5, "S"]],
                   columns=X_train.columns)

new_sex = ohe_sex.transform(new[["Sex"]])
new_embarked = ohe_embarked.transform(
    new[["Embarked"]].values)
new_age = new[["Age"]].values
new_rem = new[["Pclass", "SibSp", "Parch", "Fare"]].values

# same order as in training: rest, age, sex, embarked
new_transformed = np.concatenate(
    (new_rem, new_age, new_sex, new_embarked), axis=1)
clf.predict(new_transformed)   # 1: survives
```

4.5 What goes wrong without a pipeline

Key point: Every training step is written twice, the order must match exactly, and every change to training forces a change to the website's code.

The code above has three problems:

- **Repeated work:** every preprocessing step from training is written again in the **production code** (G-1578), the code that runs on the server.
- **Fragile order:** the 10 columns must be joined in exactly the training order. One swap gives wrong predictions without any error.
- **Changes spread:** if we change one preprocessing step in training, we must change the production code to match.

Repeating and re-ordering steps by hand is a poor way to write code, and the reason **pipelines** (G-1498) are used.

5. Building a pipeline

> **Key point:** We plan the steps, build each one as an object, and chain them with `Pipeline`.

5.1 The plan

> **Key point:** Impute, one-hot encode, scale, select the best columns, train: five steps.

The **pipeline** (G-1498) has five steps, named `trf1` to `trf5`:

1. **trf1, impute:** fill the missing Age and Embarked values, with a **column transformer** (G-415), the class that applies different transformations to different columns.
2. **trf2, one-hot encode:** encode Sex and Embarked, with a second column transformer.
3. **trf3, scale:** bring every column to the range 0 to 1 with `MinMaxScaler` (**min-max scaling** (G-1217)), so all columns are on a similar scale.
4. **trf4, feature selection:** keep only the 8 most useful of the 10 columns.
5. **trf5, model:** a **decision tree** (G-561).

Feature selection is not needed for this data. The step is included to show how such a step fits into a **pipeline** (G-1499).

The data is split exactly as before. `X_train` still has the seven raw columns.

5.2 Step 1: imputation

> **Key point:** Inside a **pipeline** (G-1498), we choose columns by position, because after the first step the data is a NumPy array without column names.

`trf1` is the same imputation as in Section 4.1, written as one **column transformer** (G-415). The columns are given by position, not by name: Age is at position 2 and Embarked at position 6.

The reason is the output. A column transformer returns a NumPy array, which has no column names.

The next step receives that array, so it cannot find a column by its name. Using positions in every step avoids this problem.

Python: Step 1.

```
from sklearn.compose import ColumnTransformer

trf1 = ColumnTransformer([
    ("impute_age", SimpleImputer(), [2]),
    ("impute_embarked",
     SimpleImputer(strategy="most_frequent"), [6]),
], remainder="passthrough")
```

`remainder="passthrough"` keeps the five other columns. Without it they would be

dropped.

5.3 Positions refer to the previous step's output

Key point: A **column transformer** (G-415) puts its transformed columns first, so after `trf1` Sex and Embarked are at positions 3 and 1, not 1 and 6.

The column transformer puts the transformed columns first and the `remainder` columns after them. So `trf1` changes the column order. Figure 4 shows how:

- **Before trf1:** Pclass, Sex, Age, SibSp, Parch, Fare, Embarked.
- **After trf1:** Age, Embarked, Pclass, Sex, SibSp, Parch, Fare.

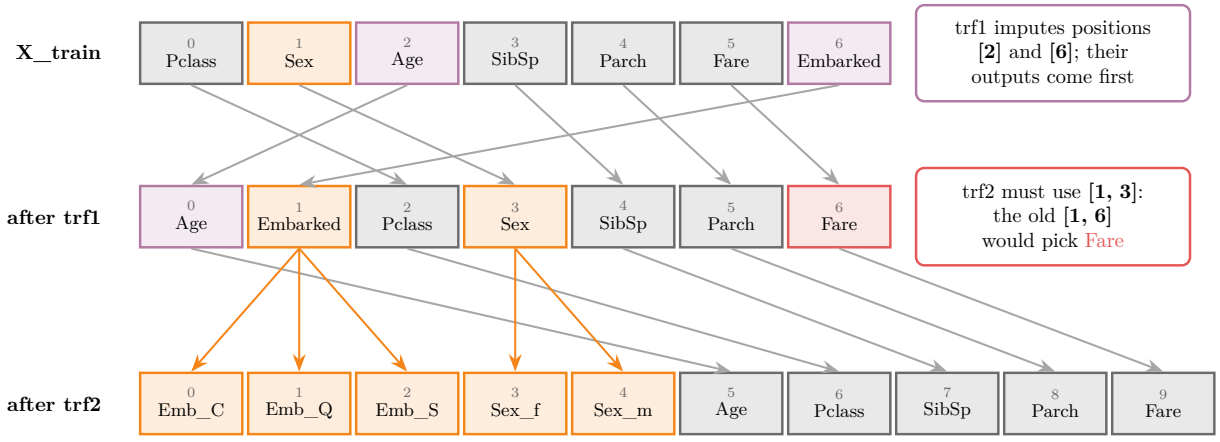


Figure 4: Column positions before and after each column transformer

`trf2` receives the reordered array. In it, Embarked is at position 1 and Sex at position 3, so `trf2` must use `[1, 3]`.

The original positions of Sex and Embarked, `[1, 6]`, would be wrong here. Position 6 is now Fare, so the encoder would one-hot encode every distinct fare. The data would grow from 10 columns to 228, and the test accuracy would drop from 78.8% to 62.6%, with no error message.

Extra: Wrong positions are a common **pipeline** (G-1498) bug. Before choosing positions for a step, check what the previous step outputs, for example with `pipe[:1].fit_transform(X_train)[:3]`. Section 10 shows a way to use column names instead.

5.4 Step 2: one-hot encoding

> **Key point:** One encoder handles Sex and Embarked together, because both are in the same array now.

Without a **pipeline** (G-1498) we needed two encoders, because Embarked and Sex lived in different arrays. Inside the pipeline both come out of `trf1` in one array, so one encoder handles both.

The output of `trf2` has 3 Embarked columns, 2 Sex columns and the 5 other columns: 10 columns.

> **Python:** Step 2.

```
trf2 = ColumnTransformer([
    ("ohe_sex_embarked",
     OneHotEncoder(sparse_output=False,
                   handle_unknown="ignore"),
     [1, 3]),    # Embarked and Sex, after trf1
], remainder="passthrough")
```

5.5 Step 3: scaling

> **Key point:** `MinMaxScaler` puts all 10 columns between 0 and 1; we use it rather than standardization because the next step needs values that are never negative.

`trf3` applies `MinMaxScaler` (normalization) to all 10 columns. Standardization would also scale the data, but it gives negative values. The **feature selection** (G-768) test in the next step only accepts values of 0 or more, so normalization is the right choice here.

The columns are given as `slice(0, 10)`: positions 0 up to, but not including, 10. The slice covers all 10 columns coming out of `trf2`.

> **Python:** Step 3.

```
from sklearn.preprocessing import MinMaxScaler

trf3 = ColumnTransformer([
    ("scale", MinMaxScaler(), slice(0, 10)),
])
```

`slice(0, 10)` is Python's object form of `0:10`.

5.6 Step 4: feature selection

> **Key point:** `SelectKBest` scores each column against the **target** (G-1949) and keeps the `k` best; it needs no **column transformer** (G-415) because it looks at every column.

Feature selection (G-768) keeps only the most useful **features** (G-772). **`SelectKBest`** (G-1762) does the job simply: it gives every column a score and keeps the `k` columns with

the highest scores.

The score here comes from `chi2`, the **chi-squared test** (G-382). The test measures how strongly each column is linked to the target; it only works on values of 0 or more (scikit-learn docs, `chi2`).

A tiny case shows what a score means. Four passengers, a 0/1 column "male" and the target survived: male = 1, 1, 0, 0 and survived = 0, 0, 1, 1. The score compares, for each group of the target, how much of the column's total falls in the group with how much a no-link split would give:

Group	Observed "male" count	Expected if there were no link
died (2 passengers)	2	1
survived (2 passengers)	0	1

$$\frac{(2-1)^2}{1} + \frac{(0-1)^2}{1} = 2$$

If "male" were 1, 0, 1, 0 instead, the observed counts would be 1 and 1, equal to the expected ones, and the score would be 0. A high score means a strong link, so the column is kept. (The computer's `chi2` gives 2 and 0 for these two cases.)

With `k=8`, two of the 10 columns are dropped.

Figure 5 shows the scores the fitted `trf4` gave the 10 columns. The two lowest, Age and Embarked_Q, are the two it drops.

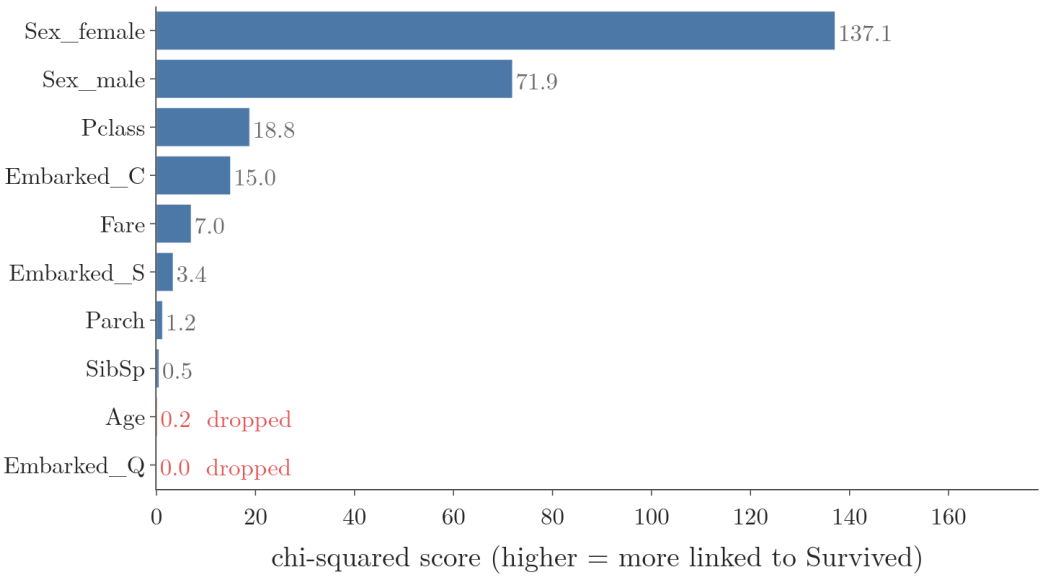


Figure 5: chi-squared score of each of the 10 columns after scaling; the 8 highest are kept

Python: Step 4.

```
from sklearn.feature_selection import SelectKBest, chi2

trf4 = SelectKBest(score_func=chi2, k=8)
```

5.7 Step 5: the model

Key point: The last step is the model itself.

Python: Step 5.

```
trf5 = DecisionTreeClassifier(random_state=42)
```

`random_state=42` makes the tree the same on every run, so our numbers can be reproduced.

5.8 Chaining the steps with Pipeline

Key point: `Pipeline` takes a list of (name, object) tuples, in the order the data should flow.

The class **`Pipeline`** (G-1498) (in `sklearn.pipeline`) takes a list of tuples. Each tuple has two items: a name we choose, and the step object. The order of the list is the order the data flows through.

Python: The **pipeline** (G-1499).

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("trf1", trf1),
    ("trf2", trf2),
    ("trf3", trf3),
    ("trf4", trf4),
    ("trf5", trf5),
])
```

5.9 Pipeline compared with make_pipeline

Key point: `make_pipeline` builds the same **pipeline** (G-1498) without asking for names; it names each step after its class.

`make_pipeline` (G-1152) is a shorter way to write the same thing: we pass only the objects. The names are made up automatically from the class names, in lower case.

Python: The same **pipeline** (G-1499) with make_pipeline.

```
from sklearn.pipeline import make_pipeline
```

```
pipe2 = make_pipeline(trf1, trf2, trf3, trf4, trf5)
pipe2.named_steps.keys()
# 'columntransformer-1', 'columntransformer-2',
# 'columntransformer-3', 'selectkbest',
# 'decisiontreeclassifier'
```

Both build the same pipeline, so the choice is a matter of taste. Names we choose ourselves, as with `Pipeline`, are shorter and easier to read when we look inside the pipeline or tune it (Sections 7 and 9).

The same pairing exists for column **transformers** (G-2009): `make_column_transformer` takes (transformer, columns) pairs and drops the name that `ColumnTransformer` asks for.

6. Training and predicting with a pipeline

> **Key point:** One `fit` call trains every step in order; one `predict` call sends new data through all the fitted steps.

Calling `pipe.fit(X_train, y_train)` sends `X_train` into `trf1`. The output of `trf1` goes into `trf2`, its output into `trf3`, then `trf4`, and finally the **decision tree** (G-561) is trained on what comes out of `trf4`. Every step learns from the training set only.

Calling `pipe.predict(X_test)` sends the test set through the same fitted steps. Nothing is learned again: each step reuses what it learned from the training set. The test accuracy is 78.8%.

Python: Training and testing the **pipeline** (G-1498).

```
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
accuracy_score(y_test, y_pred)    # 0.788
```

The accuracy is close to the 79.3% of Section 4. The two models are not identical: the **pipeline** (G-1499) also scales the data and drops two columns before the tree sees them.

6.1 One new passenger through the pipeline

> **Key point:** The same steps that processed 712 training rows process a single new row, with no extra code.

Figure 6 follows the new passenger of Section 4.4 through the fitted **pipeline** (G-1498). Each step changes the row exactly as it changed the training rows: the column order, the one-hot columns, the scaling with the training minimum and maximum, and the two dropped columns. At the end the tree predicts 0: this model says the passenger does not survive.

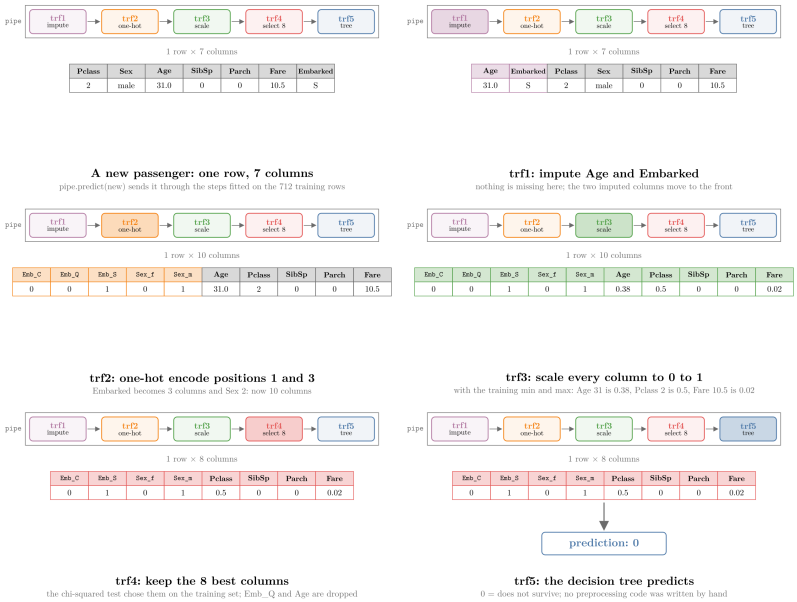


Figure 6: The new passenger flowing through the fitted pipeline, step by step

The model of Section 4 predicted 1 for the same passenger. The two trees are trained on different columns: the **pipeline** (G-1499)'s tree never sees Age or Embarked_Q (dropped by `trf4`) and sees the other columns scaled. So the two models can give different answers for the same passenger.

6.2 Pipelines without a model

> **Key point:** If the last step is a model, we call `fit` and `predict`; if the **pipeline** (G-1498) only transforms data, we call `fit_transform` and `transform`.

There are two kinds of **pipeline** (G-1499):

- **Ending with a model:** like `pipe`. We call `fit` to train it and `predict` to get predictions.
- **Only preprocessing steps:** for example, just imputation, encoding and scaling. There is nothing to predict, so we call `fit_transform` on the training set and `transform` on the test set, as with a single **transformer** (G-2009).

Python: A preprocessing-only pipeline.

```
prep = Pipeline([("trf1", trf1), ("trf2", trf2),
                 ("trf3", trf3)])
X_train_ready = prep.fit_transform(X_train)  # (712, 10)
X_test_ready = prep.transform(X_test)        # (179, 10)
```

7. Looking inside a pipeline

> **Key point:** A fitted **pipeline** (G-1498) can be drawn as a diagram, and every fitted step and its learned values can be pulled out by name.

7.1 The pipeline as a diagram

> **Key point:** In Jupyter, showing a **pipeline** (G-1498) object draws it as a diagram of its steps.

In a Jupyter Notebook, a cell that ends with `pipe` draws the **pipeline** (G-1499) as a diagram. Each box is a step, and column transformers show their inner transformers side by side with their columns. Clicking a box shows its settings.

The diagram makes it easy for anyone to see what the pipeline does:

1. impute Age and Embarked;
2. one-hot encode two columns;
3. scale;
4. select the best columns;
5. train the tree.

7.2 Pulling out fitted steps

> **Key point:** `named_steps` is a dictionary from each step's name to its fitted object.

`named_steps` (G-1301) gives every step of the **pipeline** (G-1498), under the names we chose. From a step we can go further inside, which helps when debugging.

For example, to find the mean age that the first imputer learned:

1. `pipe.named_steps["trf1"]` is the first **column transformer** (G-415).
2. Its **`transformers_`** (G-2009) attribute is the list of its fitted (name, transformer, columns) tuples. `[0]` is the first tuple, the Age imputer.
3. `[1]` takes the second item of that tuple: the fitted `SimpleImputer`.
4. Its `statistics_` attribute holds the value it fills gaps with.

Figure 7 draws the same path. Each arrow is one piece of the code below.

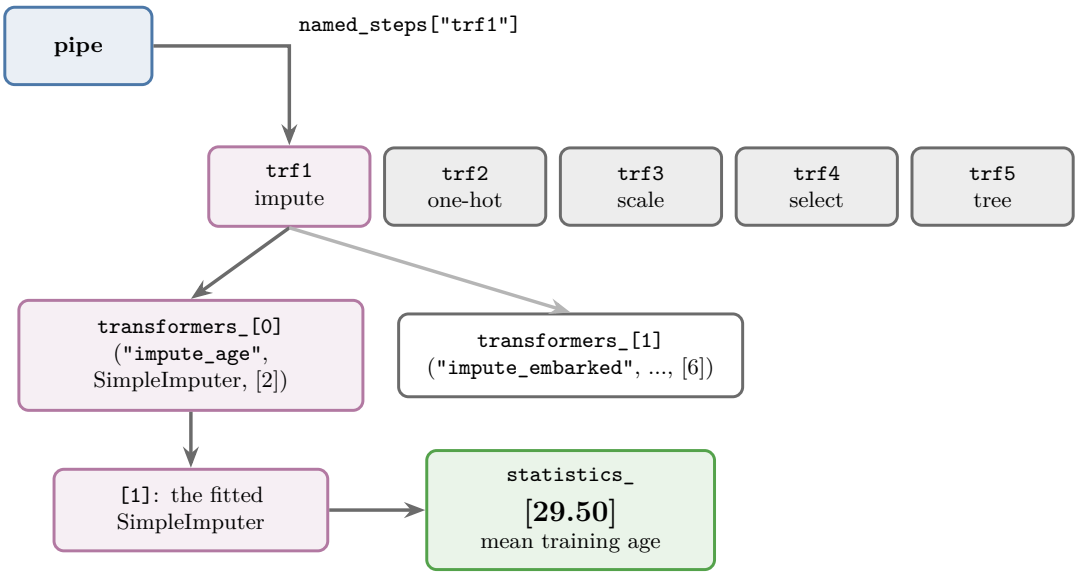


Figure 7: The path from pipe down to the mean age that the Age imputer learned

Python: Looking inside the fitted **pipeline** (G-1499).

```
pipe.named_steps["trf1"].transformers_[0][1].statistics_
# [29.50]: the mean age of the training set
pipe.named_steps["trf1"].transformers_[1][1].statistics_
# ['S']: the most frequent port
pipe.named_steps["trf4"].get_support()
# True for the 8 columns that were kept
```

8. Cross-validation with a pipeline

Key point: A whole **pipeline** (G-1498) can be passed to `cross_val_score`, exactly like a single model.

Cross-validation (G-510) tests a model more reliably than one train-test split. The training data is cut into 5 parts; the model is trained on 4 parts and tested on the fifth, five times over, so each part is the test part once.

The average of the five accuracies is the result. Here the five accuracies are 73.4, 72.7, 82.4, 80.3 and 84.5 percent:

$$73.4 + 72.7 + 82.4 = 228.5$$

$$228.5 + 80.3 + 84.5 = 393.3$$

$$\frac{393.3}{5} = 78.7$$

`cross_val_score` accepts a **pipeline** (G-1499) wherever it accepts a model. Here the average accuracy is 78.7%.

Figure 8 shows the five rounds. Watch the orange part move: in each round the whole pipeline is fitted on the four blue parts only.

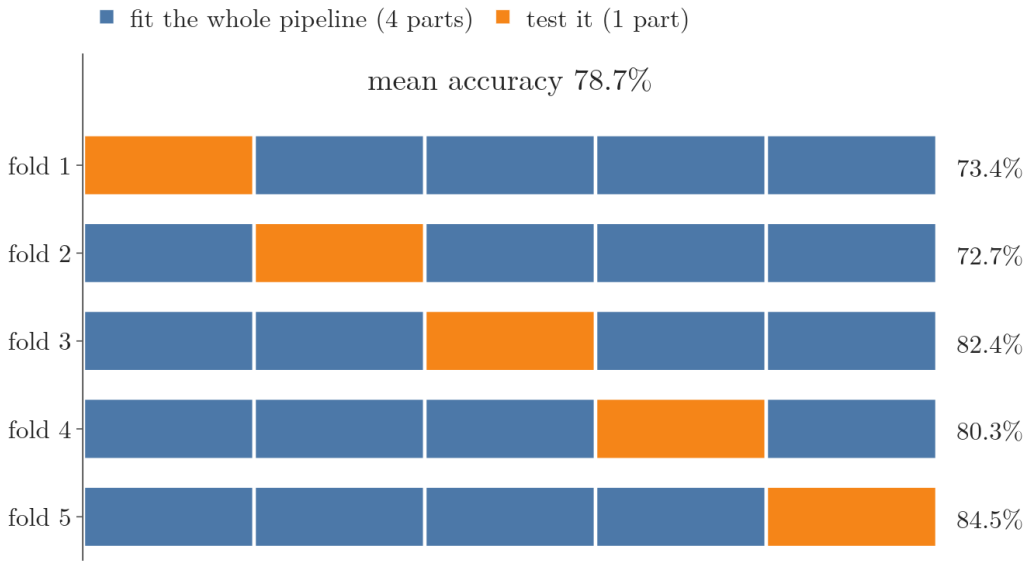


Figure 8: The five cross-validation folds of the pipeline and the test accuracy of each

Python: Cross-validating the pipeline.

```
from sklearn.model_selection import cross_val_score

cross_val_score(pipe, X_train, y_train, cv=5,
                scoring="accuracy").mean()    # 0.787
```

8.1 Why the preprocessing must be inside: data leakage

Key point: A step fitted on all the data before cross-validation has already seen every test part, so the scores come out too high. Inside a **pipeline** (G-1498), each step is refitted on the 4 training parts only.

The plain idea. A test is only honest if nothing in the model has seen the test rows. A preprocessing step that learned from the whole table has seen them.

The standard term. Information from the test rows reaching the training is called **data leakage** (G-535).

How a pipeline prevents it. Because every step is inside the **pipeline** (G-1499), every step is refitted on the 4 training parts each time. The imputer's mean, the scaler's minimum and maximum, and the chosen columns never see the part used for testing. Preprocessing the whole training set first and then cross-validating only the model would let the test part leak into training (scikit-learn User Guide, "Common pitfalls").

A worked example. On the Titanic data the leak is too small to see, so Figure 9 uses made-up data with nothing to learn: 200 **observations** (G-1374), 2,000 features of random numbers, and a random 0/1 target. On such data an honest accuracy is about 50%, the same as guessing. We keep the 20 "best" features with `SelectKBest` in two ways:

1. **Select first, then cross-validate.** `SelectKBest` is fitted on all 200 rows, and only the model is cross-validated.
2. **Selection inside the pipeline.** `SelectKBest` is a pipeline step, so each round picks its 20 **features** (G-772) from its 4 training parts only.

Watch the orange bars: selecting first gives 68% to 80% on data that holds no pattern at all. Averaged over 20 such datasets, the leaky way scores 78% and the pipeline 51%.

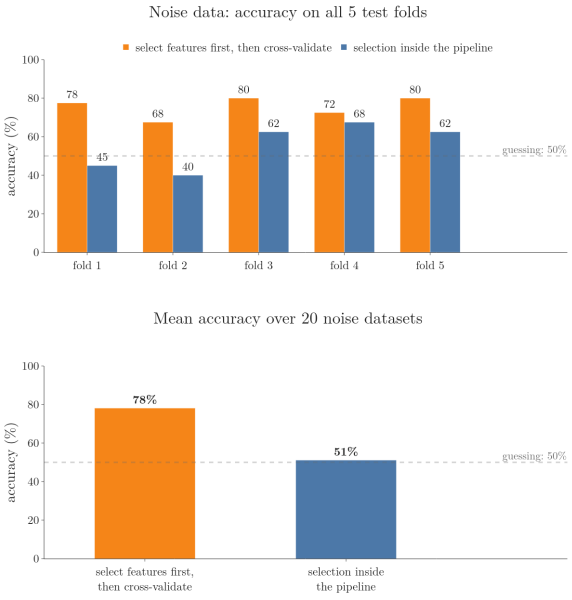


Figure 9: Cross-validation accuracy on pure-noise data: feature selection fitted before cross-validation (orange) and inside the pipeline (blue), fold by fold, then the mean over 20 datasets; experiment after the scikit-learn User Guide, "Common pitfalls"

The 78% is not skill. With 2,000 random features, some match the random **target** (G-1949) by chance on these 200 rows; choosing them with the test rows in view carries that chance match into every test part.

9. Hyperparameter tuning with a pipeline

> **Key point:** `GridSearchCV` can tune any step of a **pipeline** (G-1498); a parameter is named as the step name, two underscores, then the parameter name.

A **hyperparameter** (G-910) is a setting of an algorithm that we choose before training. A **decision tree** (G-561)'s `max_depth`, for example, limits how many questions deep the tree can grow, and changing it can make the model better or worse. **Hyperparameter tuning** (G-909) means trying several values and keeping the best.

`GridSearchCV` (G-89) does the search automatically. The search cross-validates the **pipeline** (G-1499) once for each value and keeps the value with the best average score.

For a single model, we would name the parameter just `max_depth`. In a pipeline, scikit-learn needs to know which step the parameter belongs to, so the name is **`step__parameter`** (G-146): the step's name, two underscores, then the parameter. Our tree is the step named `trf5`, so the name is `trf5__max_depth`.

Python: Tuning max_depth inside the pipeline.

```
from sklearn.model_selection import GridSearchCV

params = {"trf5__max_depth": [1, 2, 3, 4, 5, None]}
grid = GridSearchCV(pipe, params, cv=5,
                    scoring="accuracy")
grid.fit(X_train, y_train)

grid.best_score_    # 0.803
grid.best_params_   # {'trf5__max_depth': 3}
```

The best tree has a depth of 3, with an average **cross-validation** (G-510) accuracy of 80.3%. On the test set, the tuned pipeline scores 79.3%.

Figure 10 shows the score for every depth tried. Depth 3 is the peak; deeper trees score lower here, because a deeper tree keeps splitting until it fits the noise of its training parts, which does not repeat in the held-out part (**overfitting**, G-1429; see a fully grown tree overfits).

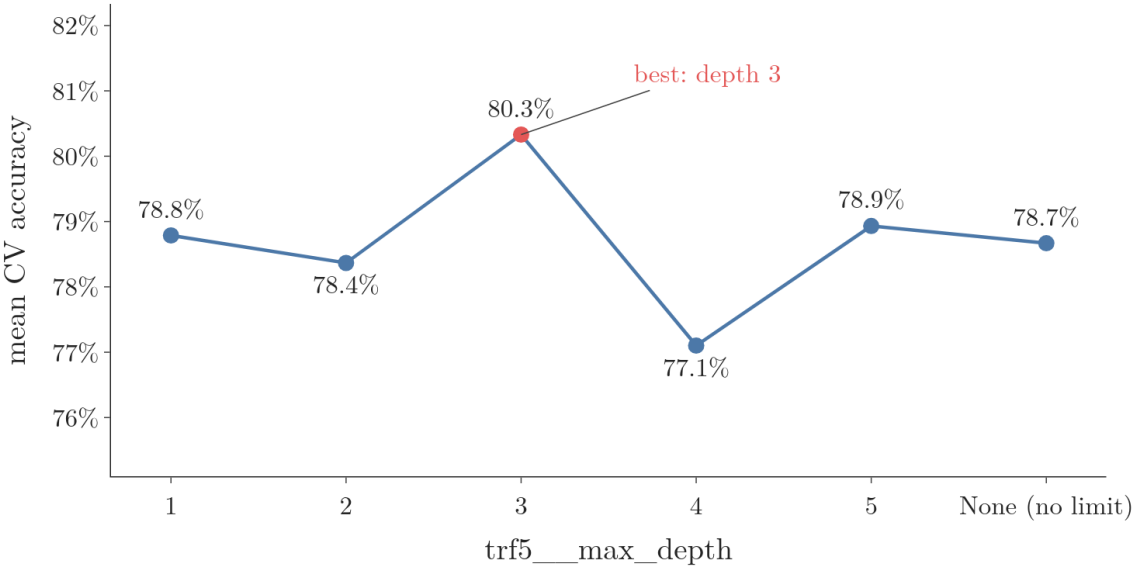


Figure 10: Mean cross-validation accuracy of the pipeline for each value of `trf5__max_depth`

> **Extra:** The same naming reaches deeper. For example, `trf4__k` tunes how many columns `SelectKBest` keeps, and `trf1__impute_age__strategy` tunes the Age imputer inside the first **column transformer** (G-415).

10. Exporting the pipeline for production

> **Key point:** One saved file holds every fitted step, so the **production code** (G-1578) is just load and predict.

We save the whole fitted **pipeline** (G-1498) to one file with pickle. The encoders and imputers do not need separate files, because everything they learned is already inside `pipe`.

> **Python:** Saving the **pipeline** (G-1499).
> ```
> pickle.dump(pipe, open("models/pipe.pkl", "wb"))
> ```

The production code loads that one file and calls `predict`. All the preprocessing of Section 4.4 disappears.

> **Python:** The website's code with a pipeline.
> ```
> import pickle
> import pandas as pd
>
> pipe = pickle.load(open("models/pipe.pkl", "rb"))
>
> new = pd.DataFrame([[2, "male", 31.0, 0, 0, 10.5, "S"]],
>     columns=["Pclass", "Sex", "Age", "SibSp",
> ```

```
                "Parch", "Fare", "Embarked"])
pipe.predict(new)    # 0: does not survive
```

Figure 11 compares the two versions of the production code.

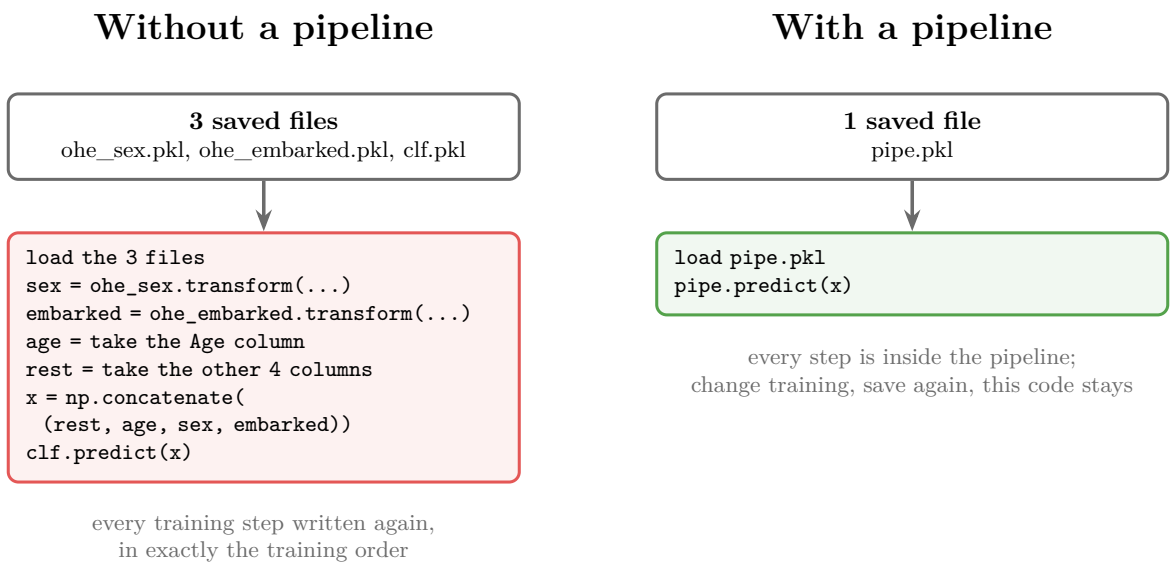


Figure 11: Production code without a pipeline (three files, every step repeated) and with one (one file, one call)

The bigger gain comes later. Suppose we change the training: add a step, remove one, or change a setting.

We save the new pipeline over `pipe.pkl` and put that file on the server. The production code does not change at all, so a change in training cannot break the website.

Extra: `joblib` does the same job as pickle with simpler calls, and it handles models holding large NumPy arrays better (scikit-learn User Guide, "Model persistence"): `joblib.dump(pipe, "models/pipe.joblib")` and `pipe = joblib.load("models/pipe.joblib")`. With either tool, only load files from sources you trust: loading a pickle file can run any code hidden in it. Load the file with the same scikit-learn version that saved it (scikit-learn User Guide, "Model persistence").

Extra: The new input is passed as a one-row DataFrame with the training column names. A bare NumPy array also works for this pipeline, because every step uses positions, but scikit-learn then warns that the input has no **feature** (G-772) names.

10.1 Using column names instead of positions

Key point: With `set_output(transform="pandas")`, every step passes a DataFrame to the next, so steps can pick columns by name.

Extra: Section 10.1 goes beyond the basics: it uses newer scikit-learn tools that remove the position counting of Section 5.3.

Positions were needed because each **column transformer** (G-415) outputs a nameless NumPy array. Calling **`set_output(transform="pandas")`** on the **pipeline** (G-1498) makes every step output a DataFrame instead, so the next step can find columns by name.

With `verbose_feature_names_out=False`, the column **transformers** (G-2009) keep plain names like `Age` and `Sex_male` instead of adding their own name as a prefix. Each step can then list its columns by name, wherever they ended up.

Python: The same **pipeline** (G-1499) with column names.

```
pipe_named = Pipeline([
    ("trf1", ColumnTransformer([
        ("impute_age", SimpleImputer(), ["Age"]),
        ("impute_embarked",
         SimpleImputer(strategy="most_frequent"),
         ["Embarked"]),
    ], remainder="passthrough",
       verbose_feature_names_out=False)),
    ("trf2", ColumnTransformer([
        ("ohe_sex_embarked",
         OneHotEncoder(sparse_output=False,
                       handle_unknown="ignore"),
         ["Sex", "Embarked"]),
    ], remainder="passthrough",
       verbose_feature_names_out=False)),
    ("trf3", MinMaxScaler()),
    ("trf4", SelectKBest(score_func=chi2, k=8)),
    ("trf5", DecisionTreeClassifier(random_state=42)),
]).set_output(transform="pandas")

pipe_named.fit(X_train, y_train)   # accuracy 0.788
pipe_named[:-1].get_feature_names_out()
# Sex_female, Sex_male, Embarked_C, Embarked_S,
# Pclass, SibSp, Parch, Fare
```

The named pipeline gives the same 78.8% accuracy, and `get_feature_names_out` shows which 8 columns reached the tree: Age and Embarked_Q were the two dropped. `pipe_named[:-1]` means every step except the last, the same slicing as for a list.

11. Summary

	Without a pipeline	With a pipeline
Preprocessing objects	one per column and task	inside one object
Joining columns	`np.concatenate` by hand	automatic
Test set	every step called again	`pipe.predict(X_test)`
Saved files	one per fitted object (3 here)	one (`pipe.pkl`)
Production code	repeats every step, in order	load and `predict`
Change to training	production code changes too	save the file again; code stays
Cross-validation and tuning	preprocessing must be redone by hand	pass `pipe` to `cross_val_score` or `GridSearchCV`

- A **pipeline** (G-1498) chains steps so each step's output is the next step's input, so the training set, the test set and every new input get exactly the same preprocessing in the same order.
- `Pipeline` takes (name, object) tuples; `make_pipeline` takes just the objects and names them itself. Our own short names are easier to read when we look inside the **pipeline** (G-1499) or tune it.
- A pipeline ending in a model uses `fit` and `predict`; a preprocessing-only one uses `fit_transform` and `transform`, because it has nothing to predict.
- Column **transformers** (G-2009) inside a pipeline usually pick columns by position, because each step outputs a NumPy array without column names, and positions refer to the previous step's output: transformed columns come first. A wrong position fails silently (encoding Fare instead of Embarked dropped the accuracy from 78.8% to 62.6%).
- `named_steps` and `transformers_` reach any fitted step and its learned values, so we can check what each step learned, such as the mean age, when debugging.
- `cross_val_score` and `GridSearchCV` accept a pipeline; a step's parameter is named `step__parameter`, because scikit-learn must know which step the parameter belongs to.
- Preprocessing fitted before **cross-validation** (G-510) leaks the test parts into training; inside a pipeline it is refitted on the training parts of every fold, so the score stays honest (on random data: 78% with the leak, 51% with the pipeline).
- One pickled pipeline is all the **production code** (G-1578) needs, so a change to training means saving the file again, and the website's code never has to repeat or re-order a step.
- So a pipeline keeps every preprocessing step and the model in one object, and the same steps run in the same order on training data, test data and every new input.

12. Sources

Built from

- CampusX, “Machine Learning Pipelines A-Z | Day 29 | 100 Days of Machine Learning”, YouTube, https://www.youtube.com/watch?v=xOccYkgRV4Q

Other references

- scikit-learn API reference. `sklearn.feature_selection.chi2`. scikit-learn.org.
- scikit-learn User Guide. Common pitfalls and recommended practices; Model persistence. scikit-learn.org.

13. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Pipeline (class) (G-1498)	The scikit-learn class (in `sklearn.pipeline`) that builds a pipeline from a list of (name, object) tuples.
Pipeline (G-1499)	One object that chains preprocessing steps and a model, each step’s output feeding the next; every input, including new ones after deployment, then gets exactly the same preprocessing.
handle_unknown=“ignore” (G-876)	`OneHotEncoder` setting that outputs all zeros for a category not seen in training.
Production code (G-1578)	The code that runs the deployed model on a server, for example behind a website.
SelectKBest (G-1762)	scikit-learn class that gives every column a score and keeps the `k` best, so only the most useful features go into the model.
Chi-squared test (chi2) (G-382)	A test scoring how strongly a column is linked to the target; needs values of 0 or more.
make_pipeline (G-1152)	scikit-learn function that chains steps such as preprocessing and a model into one object (a pipeline) from the objects alone, naming each step after its class.
transformers_ (G-2009)	The attribute of a fitted column transformer that lists its fitted (name, transformer, columns) tuples; we use it to pull out one fitted step.
named_steps (G-1301)	A pipeline’s dictionary from each step’s name to its object; used to reach one step and look inside it, for example when debugging.

Term	Meaning
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Data leakage (G-535)	Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Hyperparameter tuning (G-909)	Trying several hyperparameter values and keeping the best.
`GridSearchCV` (G-89)	The scikit-learn class that tries every combination of settings in a grid, scores each by cross-validation, and keeps the best.
`step__param` name (G-146)	The full name of a setting inside a pipeline: step names and the parameter joined by `__`.
best_score_ (G-281)	The best mean cross-validated score found by a search.
joblib (G-983)	A library that saves and loads Python objects like pickle, better suited to large arrays.
make_column_transformer (G-1150)	scikit-learn function that sets up different preprocessing for different columns (a column transformer) from (transformer, columns) pairs, without you naming the steps.
slice(0, 10) (G-1822)	Python object meaning positions 0 up to, not including, 10.
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Deployment (deploy, model deployment) (G-592)	Moving a model from development to production: putting it on a server so users can reach it.
Target (G-1949)	The output column we predict, such as the class; also called the label.

Term	Meaning
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.
Multicollinearity (G-1273)	A relationship between input columns, so that one can be calculated (or largely predicted) from the others; it makes a linear model's coefficients unstable and hard to interpret.
Accuracy (G-162)	The fraction of predictions that are correct.
Column transformer (G-415)	A scikit-learn class that applies different transformations to different columns at once and joins the results.
pickle (G-1494)	A Python module that saves objects to a file and loads them back.
Min-max scaling (G-1217)	Subtract the column's minimum and divide by its range, giving values from 0 to 1; the main normalization technique.
Feature selection (G-768)	Keeping only the useful input columns and dropping the rest.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.